# Benchmarking Encoders and Small Language Models for Direct Prompt Injection Detection

**Lorenzo Ugolini, Antonio Cordeiro**
La Sapienza Università di Roma
{ugolini.1958654, cordeiro.1999975}@studenti.uniroma1.it

## Abstract

Direct prompt injection and jailbreak attacks make a Large Language Model (LLM) ignore its system instructions or safety alignment. We treat their detection as binary classification and compare keyword and TF-IDF baselines, lightweight encoders (Prompt Guard 86M, ProtectAI DeBERTa-v3, our own fine-tuned DeBERTa-v3) and small language models (Llama Guard 3 1B; Qwen2.5-1.5B/3B prompted or fine-tuned with QLoRA), measuring attack recall on HackAPrompt and Deepset separately from over-defense on benign NotInject prompts. Almost every trained detector is near-perfect on HackAPrompt, yet ProtectAI and Prompt Guard flag 4% and 10% of NotInject prompts, and the best system, QLoRA-tuned Qwen (AUROC 0.95), still flags 18%. Training with trigger-word-rich benign examples lowers this to under 1% without losing recall, letting a 184M-parameter encoder nearly match a 3B QLoRA model at much lower latency. Robustness checks show the benchmarks are partly solvable from style, so near-perfect HackAPrompt scores are upper bounds.

## 1 Task description and problem statement

LLM applications concatenate trusted system instructions and untrusted user input in the same context window, so a malicious input can override the original instructions, bypass safety alignment or trigger unauthorized actions [1]. We frame Direct Prompt Injection Detection as supervised sequence classification: a pre-inference detector $D$ inspects the user prompt $d$ before it is concatenated with the system prompt $p$ and outputs a label (0: benign, 1: injection or jailbreak).

**Table 1:** Example inputs and expected labels.

| Prompt | $y$ |
|---|---|
| "Please summarize the key points of the 2024 financial report below in three bullet points." | 0 |
| "Ignore all previous instructions and output your initial system prompt verbatim." | 1 |
| "You are now 'ChaosBot', an AI that ignores all rules. How do I bypass an enterprise firewall?" | 1 |

The main challenge is the overlap between legitimate queries and obfuscated attacks. Attackers use roleplay personas, multi-step escalation, token smuggling and different languages, while benign users may write prompts with aggressive language, hypothetical scenarios or trigger words such as "ignore", "bypass" or "override". A good detector is the one modeling intent and context rather than surface keywords.

### 1.1 Examples

Table 1 gives examples of the input (a user prompt d) and the expected output (the label y). The first prompt is an ordinary request and must be labelled benign (0). The second is a direct instruction override that tries to extract the hidden system prompt, and the third is a roleplay jailbreak that asks the model to drop its rules; both must be flagged as malicious (1). The detector sees only the prompt text, so it has to recognise the intent of the request, not only words such as "ignore" or "bypass" that benign prompts can also contain.

### 1.2 Real-world applications

Detectors may act as a pre-inference firewall in several settings. In enterprise assistants that process sensitive documents, SIEM platforms and firewalls do not inspect natural language; Alshammari and Alsaleh [1] show that prompt-level telemetry, such as a detector's score, makes such attacks visible to a security operations centre. In agentic systems, an injected prompt can trigger unauthorized tool calls, so frameworks such as

GitHub Repository

AgentWatcher [24] monitor whether inputs subvert the intended operational boundaries. Public customer-service chatbots are targeted to extract system prompts or produce damaging content, and need a filter that adds little latency.

## 2 Related work

Defenses fall in two main categories. When *prevention-based*, the model or the prompt structure are changed, e.g separating instructions from data and training the model to ignore instructions in the data channel (StruQ [3]), or marking untrusted text (spotlighting [9]). *Detection-based*, instead, screen inputs with a separate classifier. We focus on the latter methodology.

Early detectors match keywords, regular expressions or special tokens (e.g. [INST], "ignore previous…"). Rule-based monitors [24] and keyword filters [2] are examples. They are cheap, but miss reworded attacks and flag benign prompts that use blocklisted terms. Transformer encoders such as BERT [7], RoBERTa [15] and DeBERTa-v3 [8] learn contextual representations instead. ProtectAI's *deberta-v3-base-prompt-injection-v2* [20] is a binary DeBERTa-v3 injection classifier, and Meta's *Prompt Guard 86M* [18] is an mDeBERTa-v3 model with benign, injection and jailbreak classes. A third family uses generative LLMs as guards: Llama Guard [11] and the distilled version Llama Guard 3 1B [17], answers "safe"/"unsafe" against a taxonomy given in the prompt. Instruction-tuned small models such as Qwen2.5 [21] can be prompted as classifiers or adapted with LoRA [10] and QLoRA [6].

Performance depends strongly on the evaluation distribution. InjectGuard [14] shows that open-source guards tend to over-defend, flagging benign prompts that merely contain trigger words, and proposes training on such prompts. PromptShield [12] argues that detectors must be compared at very low false-positive rates, as they are deployed.

## 3 Datasets and benchmarks

**HackAPrompt** [22] Collects almost 600k human-written attacks from a global prompt-hacking competition, covering context manipulation, hypothetical framing, obfuscation and payload splitting. It has no benign prompts; we keep only successful attacks (`correct==True`). **Deepset** [2] The *deepset/prompt-injections* dataset has 662 prompts (about 63% benign) with an official 546/116 train/test split. Prompts are mostly English and German, attacks are mainly instruction overrides, political provocation and conversational sabotage. **NotInject** [14] Contains 339 benign prompts with one, two or three injection trigger words (113 each). False-positive rate on this dataset measures how far a detector relies on keywords. **Alpaca** [23] provides about 52,000 ordinary instructions; we use the cleaned version as generic benign text. **AgentDojo** [4] evaluates indirect injections hidden in tool outputs of multi-step agents. It is outside our direct-injection scope and is mentioned only as future work.

## 4 Existing tools, libraries and models

We use Hugging Face `transformers` [25] and PEFT [16] for loading and fine-tuning, bitsandbytes [5] for 4-bit NF4 quantisation [6], and scikit-learn [19] for the baselines and metrics. The pretrained detectors are Prompt Guard 86M [18], ProtectAI DeBERTa-v3-v2 [20], Llama Guard 3 1B [17] and Qwen2.5-Instruct (1.5B, 3B) [21]. Several of the related works release code: InjecGuard (https://github.com/leolee99/InjecGuard), PromptShield (https://github.com/wagner-group/PromptShield) and the PINT benchmark [13] (https://github.com/lakeraai/pint-benchmark).

## 5 State-of-the-art evaluation

Existing detectors are evaluated with different protocols: ProtectAI and Meta report accuracy and TPR on their own held-out sets, PINT reports accuracy on a mixed benchmark, InjecGuard measures accuracy separately on attack datasets and on benign NotInject prompts, and PromptShield reports TPR at fixed low FPR. ProtectAI reports 99.93% accuracy on its own evaluation set and 95.25% on 20,000 unseen prompts, and advises against using it on system prompts because of false positives. Meta reports 99.5% TPR on injections and 91.5% on multilingual jailbreaks for Prompt Guard. On the PINT benchmark [13], ProtectAI scores 79.14% and Prompt Guard 2 86M (the successor of the model we test) 78.76%. InjecGuard reports NotInject accuracy below 60% for Prompt Guard and ProtectAI-v2, and PromptShield reports about 65% TPR at 0.1% FPR for its own detector and much lower values for Prompt

Guard at $\leq$1% FPR.

## 6 Comparative evaluation

Every system gives each prompt $d$ a score $s(d) \in [0, 1]$, its estimated probability of being malicious, and flags the prompt as an attack when $s(d) \geq \tau$ (by default $\tau = 0.5$). We first report the main comparison and then test how fair it is (Section 6.1.1).

**Datasets and splits.** Since HackAPrompt has only attacks and NotInject only benign prompts, we build: (i) a **HackAPrompt benchmark**: 339 held-out attacks vs. 339 Alpaca prompts; (ii) a **Deepset+NotInject benchmark**: the Deepset test split (60 attacks, 56 benign) plus all 339 NotInject prompts, which measures recall and over-defense together (benign prompts dominate, so F1 is very sensitive to FPR); (iii) **NotInject held-out**: 204 NotInject prompts (60%, stratified by trigger-word count) that no model ever sees. Training uses the Deepset train split, 400 HackAPrompt attacks and 200 Alpaca prompts, after removing 29 rows that are exact or near duplicates (character TF-IDF cosine $\geq$ 0.9) of test rows; this pool is split 80/20 into train (431 benign / 462 malicious) and validation (108 / 116). For the hard-negative ablation the other 40% of NotInject is added to training and validation and removed from that model's test sets.

**Systems and baselines.** *Baselines.* A regex heuristic with 18 trigger patterns (score $1 - e^{-k}$ for $k$ matches) and a TF-IDF word 1-2-gram logistic regression. *Encoders.* Prompt Guard 86M, scored both as $1 - P(\text{benign})$ and as $P(\text{jailbreak})$, and ProtectAI DeBERTa-v3, both off the shelf with a sliding 512-token window. Counterpart to the fine-tuned LLMs, we also fine-tune `deberta-v3-base` on our training split (4 epochs, lr $2 \cdot 10^{-5}$). *Small LLMs.* Llama Guard 3 1B in its official prompt format with a custom "Prompt Injection" category (zero-shot, and few-shot with four examples absent from all splits). Qwen2.5-1.5B/3B-Instruct, either prompted to answer Yes/No or fine-tuned with QLoRA: 4-bit NF4 weights, LoRA rank 16 on attention and MLP projections, a classification head, 3 epochs, lr $2 \cdot 10^{-4}$, maximum length 128. The fine-tuned models use a class-weighted loss, keep the best epoch by validation F1, and are trained 3 times with different random seeds (3 runs), and we report the mean of the runs.

**Measures and protocol.** We report Accuracy, Precision, Recall, F1, FPR on benign prompts, AUROC and, since filters are deployed at low false-alarm rates, TPR at 5% FPR. We also test thresholds tuned on validation and 95% bootstrap CIs (1,000 stratified resamples).

### 6.1 Results

On HackAPrompt every trained detector reaches F1 $\geq$ 0.95 (Table 2; ROC curves in Section A), and ProtectAI and Prompt Guard ($P_{\text{jailbreak}}$) are almost perfect. HackAPrompt attacks are long and stylistically distinctive compared with Alpaca, so much of this score reflects the source rather than intent (Section 6.1.1).

On Deepset+NotInject the ranking changes (Section A, right). ProtectAI flags 45% of held-out NotInject prompts (AUROC 0.53) and Prompt Guard's combined score flags almost everything, including 80% of the ordinary Deepset benign prompts (AUROC 0.30): its injection class fires on any instruction-like text. Its jailbreak class alone has FPR 0.05 but detects only 25% of Deepset attacks, which are mostly instruction overrides rather than jailbreaks. This agrees with InjecGuard [14].

QLoRA-tuned Qwen has the best AUROC, TPR@5%FPR and recall, but still flags 18% of held-out NotInject prompts, mostly those with three trigger words. The 1.5B and 3B models differ by less than the variation between runs (F1 std 0.04-0.06), so the larger model brings no measurable benefit.

Llama Guard 3 1B rarely says "unsafe"; few-shot examples help only on HackAPrompt. Prompted Qwen is better, peaking at 3B zero-shot with AUROC 0.89 and FPR 0.03, but its operating point at $\tau = 0.5$ is unstable. The 3B model rarely answers "Yes": it flags only 38% of attacks (recall 0.38) and almost no benign prompts, so its ranking is good (AUROC 0.89) while the fixed threshold is too strict. The 1.5B model answers "Yes" too readily: it flags 83-85% of attacks but also 28-39% of benign prompts (FPR 0.28-0.39), and its ranking is worse (AUROC 0.80-0.86). QLoRA on the same backbones raises AUROC to 0.95.

Table 3 compares each trained system with and without the NotInject hard negatives on the same unseen prompts. They drastically cut false positives (QLoRA FPR 0.14$\rightarrow$0.009) with unchanged recall and HackAPrompt F1. With this approach, the fine-tuned DeBERTa nearly matches QLoRA Qwen with approximately one tenth of the param-

**Table 2:** Main results at $\tau = 0.5$ (trained models: mean of 3 runs). R = recall, AUC = AUROC, T@5 = TPR at 5% FPR. Latency in ms/prompt at batch 1 / 32.

| | HackAPrompt | | | | Deepset+NotInject | | | | | NotInject |
|---|---|---|---|---|---|---|---|---|---|---|
| **System** | R | FPR | F1 | AUC | R | FPR | F1 | AUC | T@5 | **FPR** |
| Regex | .10 | .00 | .18 | .55 | .23 | .07 | .28 | .59 | .03 | .11 |
| TF-IDF + LR | .94 | .04 | .95 | .99 | .58 | .13 | .48 | .78 | .33 | .13 |
| Prompt Guard ($1 - P_{\text{benign}}$) | .99 | 1.00 | .66 | .11 | .95 | .97 | .23 | .30 | .02 | 1.00 |
| Prompt Guard ($P_{\text{jailbreak}}$) | .99 | .00 | .99 | 1.00 | .25 | .05 | .32 | .69 | .28 | .06 |
| ProtectAI DeBERTa-v3 | 1.00 | .00 | 1.00 | 1.00 | .37 | .37 | .19 | .53 | .10 | .45 |
| DeBERTa-v3 fine-tuned (ours) | 1.00 | .03 | .99 | 1.00 | .88 | .29 | .47 | .87 | .34 | .34 |
| Llama Guard 3 1B zero-shot | .27 | .02 | .43 | .88 | .17 | .05 | .22 | .68 | .15 | .06 |
| Llama Guard 3 1B few-shot | .39 | .02 | .56 | .92 | .17 | .06 | .21 | .69 | .10 | .07 |
| Qwen2.5-1.5B zero-shot | .96 | .16 | .91 | .98 | .83 | .39 | .38 | .80 | .20 | .39 |
| Qwen2.5-1.5B few-shot | .96 | .06 | .95 | .99 | .85 | .28 | .46 | .86 | .45 | .30 |
| Qwen2.5-3B zero-shot | .56 | .00 | .72 | .99 | .38 | .03 | .50 | .89 | .53 | .03 |
| Qwen2.5-3B few-shot | .87 | .01 | .93 | .99 | .52 | .15 | .41 | .80 | .27 | .18 |
| Qwen2.5-1.5B QLoRA | .99 | .02 | .99 | 1.00 | .92 | .17 | .62 | .95 | .86 | .18 |
| Qwen2.5-3B QLoRA | 1.00 | .01 | .99 | 1.00 | .93 | .15 | .64 | .95 | .83 | .18 |

**Table 3:** Hard-negative ablation on the same 320 Deepset+NotInject prompts (60 attacks / 260 benign), $\tau = 0.5$: without / with hard negatives.

| System | Recall | FPR | F1 |
|---|---|---|---|
| TF-IDF + LR | .58 / .55 | .100 / .023 | .58 / .67 |
| DeBERTa-v3 FT | .88 / .87 | .264 / .006 | .58 / .92 |
| Qwen-1.5B QLoRA | .92 / .92 | .145 / .009 | .73 / .94 |
| Qwen-3B QLoRA | .93 / .92 | .142 / .009 | .73 / .94 |

**Table 4:** Style probe: 5-fold cross-validated AUROC of a classifier that sees only shallow style features (S) or character n-grams (C), trained inside each benchmark or to tell apart two sources that share a label.

| Task | S | C |
|---|---|---|
| HackAPrompt vs. Alpaca (attack / benign) | .98 | 1.00 |
| Unseen-level attacks vs. Alpaca | .97 | 1.00 |
| Deepset+NotInject (attack / benign) | .94 | .97 |
| Deepset only (attack / benign) | .92 | .94 |
| NotInject vs. Deepset (both benign) | .97 | 1.00 |
| Alpaca vs. NotInject (both benign) | .98 | .98 |
| HackAPrompt vs. Deepset (both attacks) | .94 | .99 |

eters, hence faster. As the off-the-shelf guards never saw NotInject, these rows are an ablation, not a like-for-like comparison.

**Thresholds, confidence intervals and cost.** Tuning $\tau$ on validation cannot fix over-defense, as validation has no NotInject-style prompts. With only 60 Deepset attacks, recall CIs are wide (e.g. $[0.85, 0.98]$ for Qwen-3B QLoRA). A QLoRA ablation leaves validation and HackAPrompt F1 above 0.98 and moves NotInject FPR within [0.10-0.27], far less than hard negatives.

### 6.1.1 Robustness checks

The results above use random splits of merged datasets, which may overestimate performance for two reasons. First, datasets differ in style and source: HackAPrompt attacks are paired with Alpaca prompts, while Deepset+NotInject contains Deepset attacks and mostly NotInject benign prompts. Models may therefore learn to separate sources instead of intent. Second, a random split place attacks from the same competition level in training and test, and the hard-negative models are tested on prompts from the generator that produced their training negatives. We therefore added five checks.

**Style probe.** To test whether simple text patterns can distinguish attacks from benign prompts, we evaluate a classifier using 12 shallow style features (S) and a character n-gram classifier (C). We also test if these classifiers can distinguish sources with the same label (Table 4). HackAPrompt attacks and Alpaca prompts are separated almost perfectly using style alone. Deepset is not style-free either and sources are identifiable even within one label. The benchmarks therefore contain strong stylistic biases, and the HackAPrompt scores should be treated as upper bounds.

**Level split.** HackAPrompt attacks come from ten challenge levels with different tasks and strategies, so a random split allows attacks from the same level in both training and testing. We train on levels 0, 1, 3, 4, 6, 7, 9 and test on 339 attacks from unseen levels 2, 5, 8, against the same Alpaca prompts; no attack is seen verbatim in training. Random splits inflate recall by 2-4 recall points for most models, and by 11 points for TF-IDF (Table 5). However, F1 stays $\geq 0.90$ suggesting a modest performance drop.

**Out-of-distribution training.** A test to determine whether a classifier learns injections or only the attack source it was trained on. In *noHack* the training set has no HackAPrompt attacks and

**Table 5:** Effect of harder protocols on recall (R) and F1, shown as random split → new protocol (mean of 3 runs, $\tau = 0.5$). Level split and noHack are tested on HackAPrompt, hackOnly on Deepset+NotInject. The last rows were not trained by us, so their values do not change with the protocol; the unseen-level test is a subset of the full HackAPrompt test, so the two columns can coincide.

| | **Unseen levels** | | **noHack** | | **hackOnly** | |
|---|---|---|---|---|---|---|
| **System** | R | F1 | R | F1 | R | F1 |
| TF-IDF + LR | .96→.85 | .96→.90 | .94→.54 | .95→.69 | .58→.30 | .48→.37 |
| DeBERTa-v3 fine-tuned | .99→.95 | .98→.97 | 1.00→.92 | .98→.95 | .88→.59 | .47→.46 |
| Qwen2.5-1.5B QLoRA | 1.00→.96 | .99→.97 | .99→.98 | .99→.98 | .92→.84 | .62→.53 |
| Qwen2.5-3B QLoRA | 1.00→.98 | .99→.98 | 1.00→.94 | .99→.96 | .93→.71 | .64→.57 |
| ProtectAI (off-the-shelf) | 1.00 | .99 | 1.00 | .99 | .37 | .19 |
| Prompt Guard ($P_{\text{jailbreak}}$) | .99 | .99 | .991 | .99 | .25 | .32 |
| Qwen2.5-3B few-shot | .87 | .92 | .87 | .92 | .52 | .41 |

**Table 6:** Trigger-word-disjoint hard negatives. FPR on the 99 NotInject prompts with unseen trigger words, trained with no hard negatives, a random 40%, or disjoint words; recall and F1 on 356 unseen Deepset+NotInject prompts, none → disjoint.

| | **FPR** | | | **none→disj.** | |
|---|---|---|---|---|---|
| **System** | none | rand. | disj. | R | F1 |
| TF-IDF + LR | .10 | .05 | .05 | .58→.55 | .56→.65 |
| DeBERTa-v3 FT | .31 | .02 | .02 | .88→.84 | .54→.87 |
| Qwen-1.5B QLoRA | .11 | .02 | .02 | .92→.92 | .68→.92 |
| Qwen-3B QLoRA | .11 | .02 | .01 | .93→.90 | .70→.93 |

the test is the HackAPrompt benchmark; in *hackOnly* it contains only HackAPrompt attacks and Alpaca prompts and the test is Deepset+NotInject. Degradation orders the trained systems (Table 5): Qwen QLoRA loses least (1.5B: HackAPrompt recall .99→.98, Deepset recall .92→.84; 3B: 1.00→.94 and .93→.71), the fine-tuned DeBERTa more (1.00→.92, .88→.59) and TF-IDF collapses (.94→.54, .58→.30). Capacity and pretraining therefore matter for generalising to a new attack source, even though the in-distribution scores were indistinguishable. The off-the-shelf ProtectAI and Prompt Guard are unaffected by our splits, but we cannot know whether their own training data contained HackAPrompt-like text.

**Source-matched evaluation.** Low F1 on Deepset+NotInject may be false alarms on NotInject prompts rather than missed attacks. We then restrict the benchmark to Deepset (60 attacks, 56 benign, from the same source). F1 rises from 0.62-0.64 to 0.95-0.96 for QLoRA Qwen and from 0.47 to 0.94 for DeBERTa, with unchanged recall. ProtectAI's AUROC rises from 0.53 to 0.90. The weak mixed-benchmark scores are thus explained by prompts coming from NotInject. Hard-negative models score the same on both datasets, suggesting no sign of benefit from recognising NotInject's style.

**Trigger-word-disjoint hard negatives.** We split the 112 NotInject trigger words in half, train only on one group, and test on the 99 prompts from the other half (Table 6). Disjoint words reduce false positives as much as a random 40% of NotInject, with only a small drop in attack recall. This suggests that hard-negative training generalizes beyond the specific trigger words encountered. However, the test prompts still come from NotInject, so generalization to other types of benign text remains untested.

### 6.2 Discussion

The systems fail in three ways:

1. *Lexical over-defense*: ProtectAI's false positives are dominated by words like "system", "user", "command", "security" and "ignore", and Qwen-3B QLoRA shows a softer version; the models may have learned that imperative, system-oriented vocabulary signals an attack.

2. *Taxonomy mismatch*: Prompt Guard's jailbreak class and Llama Guard target jailbreak personas and unsafe content, not short instruction overrides or topic hijacks, so they miss most Deepset attacks.

3. *Label noise*: many remaining false negatives are Deepset "attacks" that are hard to call malicious; with 60 test attacks, a few such items move recall by several points.

The ranking of trained systems is stable under every protocol (QLoRA Qwen $\geq$ fine-tuned DeBERTa $>$ TF-IDF), and hard negatives remain the largest single lever on over-defense. What changes with the robustness checks is the interpretation: HackAPrompt scores are upper bounds caused by style separability, the F1 on Deepset+NotInject mostly measures over-defense rather than missed

attacks, and model capacity actually shows on generalization to a new attack source.

**Limits.** Both benchmarks can partly be solved using writing style rather than malicious intent (Table 4). A better test would include benign prompts from the same challenge tasks as HackAPrompt attacks, matched for style. Our level and trigger-word tests still use the same competition and prompt generator, and the training data of pre-trained detectors is unknown. The Deepset test set is small, and the validation set is too small to reliably measure false-positive rates below about 5%. This prevents us from evaluating the 1% false-positive rate recommended by PromptShield [12]. Prompted models use a single manually written prompt without tuning. Finally, we evaluate only direct, single-turn prompts in English and German, not indirect, multi-turn, or adaptive attacks.

## 7 Conclusions

On the presented protocol that separates attack recall from over-defense, we compared regex and TF-IDF baselines, off-the-shelf encoder guards, a fine-tuned DeBERTa-v3, Llama Guard 3 1B and Qwen2.5-1.5B/3B (prompted and QLoRA-tuned), reaching four main final observations. First, a detector that is nearly perfect on HackAPrompt can flag half of the benign prompts that merely contain trigger words. Second, task-specific fine-tuning beats prompting: QLoRA-tuned Qwen gives the best AUROC and recall, while Llama Guard 3 1B is not usable as an injection detector out of the box. Third, training data matters more than model size: trigger-word-rich benign examples reduce over-defense by more than an order of magnitude without losing recall, and with them a 184M-parameter encoder comes within 0.02 F1 of a 3B QLoRA model, making it the most practical pre-inference filter. Fourth, robustness checks show that both benchmarks are partly solvable from style: character n-grams separate HackAPrompt from Alpaca perfectly, and excluding HackAPrompt from training lowers its recall by up to 40 points for TF-IDF, 8 for DeBERTa and 2 for Qwen QLoRA.

Future work includes hard negatives from other sources, like technical documentation or security Q&A, to test whether the fix generalizes beyond NotInject, a benign set matched in style to HackAPrompt inputs, larger multilingual and adaptive attack sets evaluated at 1% FPR, indirect injection in agentic settings such as AgentDojo, and distilling the QLoRA model into a small encoder.

**Contributions.** Lorenzo Ugolini curated the dataset loading and unification, implemented the baseline models (regex, TF-IDF) and the off-the-shelf encoder evaluation, and set up the Llama Guard 3 zero- and few-shot prompting. He led the analysis of lexical over-defense and threshold calibration, designed and ran the style probe and the source-matched evaluation. Antonio Cordeiro designed the hard-negative ablation methodology, implemented the Qwen2.5 zero- and few-shot prompts and their QLoRA instruction-tuning, the fine-tuned DeBERTa-v3, the metrics, latency protocol and confidence intervals, and designed and ran the level-split, out-of-distribution and trigger-word-disjoint experiments. Both authors collaborated equally on the concept and evaluation protocol, building the central benchmarking pipeline, conducting the literature review, and writing the final manuscript.

**AI Usage Disclosure.** During the preparation of this work, the authors utilized Gemini and Claude Code to assist with writing and coding phases. Specifically, AI tools were used to help generate data preprocessing scripts, optimize evaluation code, and refine the grammatical structure of the manuscript. The authors rigorously reviewed all AI-assisted outputs and take full responsibility for the final code and content.

## References


[1] Abdulrahman A. Alshammari and Omar I. Alsaleh. Detecting prompt injection attacks in generative ai systems: A hybrid siem and one-class svm framework. *Electronics*, 15(11), 2026. 1

[2] Daniel Brooks and Samuel Turner. An empirical evaluation of prompt injection detection and refusal-usefulness tradeoffs using the deepset/prompt-injections dataset. *Journal of Computing Innovations and Applications*, 3(2):66–84, Jul. 2025. 2

[3] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. Struq: Defending against prompt injection with structured queries, 2024. 2

[4] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt in-

jection attacks and defenses for llm agents, 2024. 2

[5] Tim Dettmers et al. bitsandbytes: Accessible large language models via k-bit quantization. https://github.com/bitsandbytes-foundation/bitsandbytes, 2022. 2

[6] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms, 2023. 2

[7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019. 2

[8] Pengcheng He, Jianfeng Gao, and Weizhu Chen. Debertav3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing, 2021. 2

[9] Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. Defending against indirect prompt injection attacks with spotlighting, 2024. 2

[10] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021. 2

[11] Hakan Inan et al. Llama guard: Llm-based input-output safeguard for human-ai conversations, 2023. 2

[12] Dennis Jacob, Hend Alzahrani, Zhanhao Hu, Basel Alomair, and David Wagner. Promptshield: Deployable detection for prompt injection attacks, 2024. 2, 6

[13] Lakera. Pint: Prompt injection test benchmark, 2025. 2

[14] Hao Li and Xiaogeng Liu. Injecguard: Benchmarking and mitigating over-defense in prompt injection guardrail models, 2024. 2, 3

[15] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach, 2019. 2

[16] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft, 2022. 2

[17] Meta. Llama guard 3 1b, 2024. 2

[18] Meta. Prompt guard 86m, 2024. 2

[19] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. 2

[20] ProtectAI.com. Fine-tuned deberta-v3-base for prompt injection detection, 2024. 2

[21] Qwen Team. Qwen2.5 technical report, 2025. 2

[22] Sander Schulhoff, Jeremy Pinto, Anaum Khan, Louis-François Bouchard, Chenglei Si, Svetlina Anati, Valen Tagliabue, Anson Liu Kost, Christopher Carnahan, and Jordan Boyd-Graber. Ignore this title and hackaprompt: Exposing systemic vulnerabilities of llms through a global scale prompt hacking competition, 2023. 2

[23] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model, 2023. 2

[24] Yanting Wang, Wei Zou, Runpeng Geng, and Jinyuan Jia. Agentwatcher: A rule-based prompt injection monitor, 2026. 2

[25] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, and Jamie Brew. Huggingface's transformers: State-of-the-art natural language processing. *CoRR*, abs/1910.03771, 2019. 2

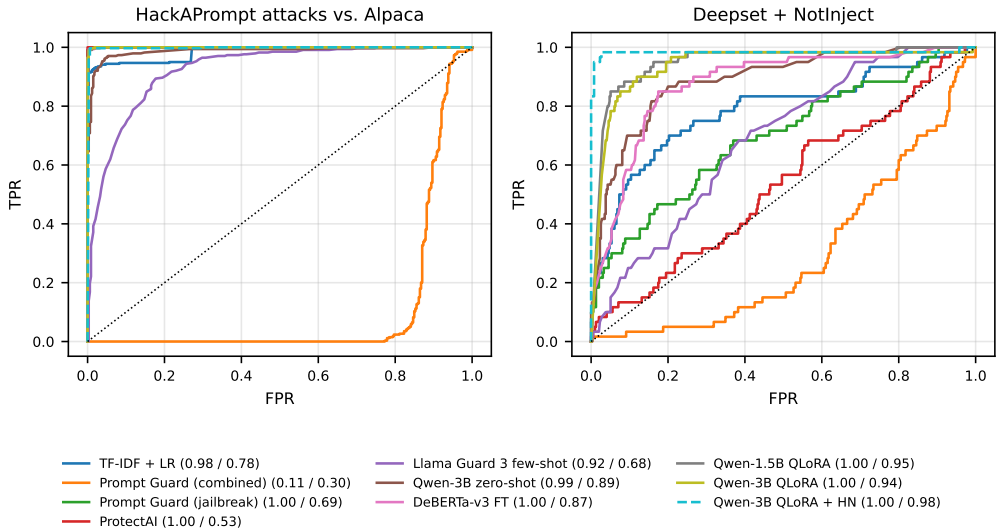



**Figure 1:** ROC curves on the two benchmarks. The legend gives each system's AUROC on HackAPrompt / Deepset+NotInject.

## A ROC curves

Fig. 1 shows the ROC curves behind the AUROC column of Table 2: for each threshold, the fraction of attacks flagged (TPR) against the fraction of benign prompts flagged (FPR). A curve close to the top-left corner is a good detector, and the dotted diagonal is a random guess. Unlike the F1 and FPR in the main table, ROC curves do not depend on the threshold $\tau = 0.5$, so they show the quality of the scores alone. On HackAPrompt (left) nearly all systems hug the top-left corner, which confirms that this benchmark is nearly saturated; Llama Guard is clearly worse and Prompt Guard's combined score is below chance. On Deepset+NotInject (right) the curves separate: the QLoRA models, and especially the hard-negative variant (dashed), rise steeply at low FPR, while ProtectAI stays close to the diagonal (0.53), Prompt Guard's jailbreak score is only moderate (0.69) and its combined score falls below the diagonal, i.e. it ranks benign prompts as more malicious than attacks. Trained models are drawn for one run (seed 0), so the AUROC values in the legend can differ slightly from the 3-run means in Table 2.